BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

INTRODUCTION TO WEB PROGRAMMING

Unit 9

Data Representation in Web

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3-4
	1.1 Learning Objectives	-	-	
2	Overview of JSON	-	-	5-8
3	Syntax	-	-	9-12
4	Data Types	-	-	13-15
5	Parsing	-	-	16-18
6	Stringify	-	-	18-20
7	Summary	-	-	21
8	Questions	-	1	22
9	Answers	-	-	23

1. INTRODUCTION

In the digital age, data exchange and communication between various applications and systems are fundamental aspects of modern computing. JSON, which stands for JavaScript Object Notation, has emerged as a cornerstone technology in this landscape. This chapter provides a comprehensive introduction to JSON, offering a deep dive into its key aspects and functionalities.

JSON is a lightweight, human-readable data interchange format that has gained widespread popularity due to its simplicity and versatility. In this chapter, we will start with an overview of JSON, exploring its origins and the fundamental role it plays in web development, data storage, and API communication. We will delve into the syntax of JSON, breaking down the components of JSON objects and arrays, and understanding the rules that govern their structure. Data types in JSON, such as strings, numbers, booleans, objects, and arrays, will be thoroughly examined, shedding light on how to represent various kinds of information effectively. Additionally, we will explore the vital processes of parsing JSON data, which allows applications to convert JSON text into usable data structures, and stringifying, which is the reverse process of converting data structures back into JSON text. This chapter serves as a solid foundation for anyone looking to work with JSON, whether they are web developers, data engineers, or software architects.

1.1 Learning Objectives

By the end of this chapter you will be able to:

- *Demonstrate an understanding of the fundamental concepts of JSON, including its origins, purpose, and key characteristics, as well as the syntax rules governing JSON data structures.*
- *Explain the various data types available in JSON, such as strings, numbers, booleans, objects, and arrays, and articulate how these data types can be used to represent different types of information effectively.*
- *Apply their knowledge of JSON by parsing JSON data into usable data structures within programming languages like JavaScript, Python, or Java, allowing them to work with JSON data in real-world scenarios.*

- *Critically evaluating JSON data structures, identifying the relationships between JSON objects and arrays, and making informed decisions about the most suitable JSON representation for specific data sets.*
- *Demonstrate their ability to stringify data structures into JSON format, effectively preparing data for transmission or storage, and integrate this knowledge into the development of robust, data-driven applications and systems.*

INSPIRED BY LIFE

2. OVERVIEW OF JSON

JSON, short for JavaScript Object Notation, is a lightweight data interchange format that has become a standard for data transmission between a server and a web application, as well as for storing structured data. JSON is a text-based format that is easy for both humans to read and write and machines to parse and generate. It was derived from JavaScript, but it is language-independent, which means it can be used with virtually any programming language. JSON is primarily used for representing structured data and is commonly used in web applications and APIs (Application Programming Interfaces). JSON, JavaScript Object Notation, is a versatile and widely adopted data interchange format that emerged as a simpler and more readable alternative to XML (Extensible Markup Language). JSON is rooted in JavaScript but has evolved into a language-independent format, making it accessible to a broad range of programming languages and platforms.

2.1 Overview and Applications of JSON

1. *Lightweight:* JSON's design emphasizes simplicity and efficiency. It uses a minimalistic syntax that reduces data size, making it an excellent choice for transmitting data over networks with low latency.
2. *Human-Readable:* JSON is designed to be easily read and written by humans. It uses a text-based format with straightforward conventions for representing data, which is a key advantage for developers and data analysts.
3. *Language-Independent:* JSON's language-agnostic nature means it can be utilized in various programming languages, from JavaScript, Python, and Java to Ruby and beyond. This interoperability enables seamless communication between different systems and applications.
4. *Data Structure:* JSON can represent a wide range of data structures, including objects, arrays, strings, numbers, booleans, and null values. This flexibility allows it to handle complex data hierarchies and nested structures.

2.2 Applications

1. *Web APIs*: JSON plays a pivotal role in modern web development, especially in the context of web APIs. When web services and applications need to exchange data, JSON

is often chosen as the format for data transmission due to its simplicity and compatibility with JavaScript, which is commonly used in web development.

2. *Configuration Files:* JSON's readability and simplicity make it a popular choice for configuration files in various software applications. Developers use JSON to store settings, preferences, and options because it's easy to understand and edit.
3. *Data Storage:* JSON is frequently employed for storing semi-structured or unstructured data in databases or data files. It excels in situations where data formats may evolve over time since JSON can accommodate changes to data structures without breaking compatibility.
4. *Serialization:* Serialization is the process of converting complex data structures from programming languages into a format that can be easily transmitted or stored. JSON is a preferred format for serializing data, allowing systems to communicate efficiently.
5. *Data Interchange:* JSON facilitates data exchange between different systems and platforms, making it a valuable tool for integrating disparate software components. It simplifies the process of sending and receiving data between applications, making it a cornerstone in system integration and interoperability.

In summary, JSON's appeal lies in its simplicity, versatility, and wide-ranging applications. It's particularly well-suited for scenarios involving data exchange in web development, storage of flexible data structures, configuration management, and system integration due to its language-agnostic nature and human-readable format.

2.3 Advantages of JSON

1. *Human-Readable:* JSON's human-readable format is one of its most prominent advantages. It uses a simple and intuitive syntax that resembles data structures in many programming languages. This readability is highly beneficial for developers and data analysts who often need to inspect or modify JSON data directly. It aids in debugging, troubleshooting, and understanding the structure of the data without the need for specialized tools or parsers.
2. *Lightweight:*JSON's simplicity extends to its lightweight data representation. The syntax is concise and devoid of unnecessary elements, which results in smaller data payloads. This compactness is crucial for efficient data transmission, particularly over networks with limited bandwidth or high latency. Smaller data sizes lead to faster data

transfers, reducing both bandwidth usage and response times in web applications and APIs.

3. *Language-Independent:* JSON's language independence is a key feature that contributes to its versatility and wide adoption. JSON can be used with almost any programming language, from JavaScript, Python, and Ruby to Java, C#, and PHP. This compatibility eliminates language barriers, allowing different systems and platforms to exchange data seamlessly. Developers can serialize data in one language and deserialize it in another, facilitating interoperability in complex software ecosystems.
4. *Data Structure Flexibility:* JSON is highly flexible in representing a wide range of data structures. It supports various data types, including strings, numbers, booleans, objects, arrays, and null values. JSON's ability to nest objects and arrays within one another enables the representation of complex and hierarchical data structures. This flexibility makes it suitable for modeling diverse data types, from simple configurations to intricate data relationships.
5. *Widely Supported:* JSON enjoys extensive support across programming languages and libraries. Many programming languages provide built-in support for parsing and generating JSON, simplifying the integration of JSON data into applications. Additionally, numerous third-party libraries and frameworks offer JSON-related functionality, such as validation, transformation, and manipulation. This wide support ecosystem ensures that JSON can be easily adopted and integrated into diverse software projects.

In summary, JSON's advantages encompass its human-readability, lightweight data representation, language independence, flexibility in handling data structures, and widespread support across programming languages and ecosystems. These qualities make JSON an ideal choice for a wide range of applications, particularly in web development, data interchange, configuration management, and data storage.

2.4 Disadvantages of JSON

1. *Lack of Schema:*

JSON does not come with a built-in schema definition, unlike some other data formats like XML or databases with predefined schemas. This means there is no standardized way to enforce data structure constraints, such as required fields, data types, or relationships

between data elements. Without a schema, it falls on developers to ensure that JSON data adheres to the expected structure. This can lead to data integrity issues if not managed carefully, as unexpected or malformed data may be difficult to detect and handle properly.

2. *Limited Data Types:* JSON has a relatively simple set of native data types, including strings, numbers, booleans, objects, arrays, and null. While these data types cover many common use cases, JSON lacks native support for more specialized or complex data types like dates, binary data, or custom enumerations. When dealing with such data, developers need to implement custom serialization and deserialization logic, which can introduce complexity and potential compatibility issues.
3. *No Comments:* JSON does not support comments within its data structures. Comments are often used in data files or configuration files to provide explanatory notes, document the purpose of specific fields, or disable certain sections temporarily. The absence of comments can make JSON less expressive for documentation purposes. Developers are forced to rely on external documentation or conventions to provide context for the data, which can be less convenient and prone to being out of sync with the actual data.
4. *Security Concerns:* When parsing JSON data from untrusted or potentially malicious sources, there are security concerns to consider. One notable vulnerability is JSON injection, similar to SQL injection but specific to JSON. If not properly sanitized and validated, user-provided JSON data can be manipulated to execute unintended actions or access unauthorized data within an application. To mitigate these risks, developers must implement stringent input validation and sanitation procedures when handling external JSON data.

In conclusion, JSON's disadvantages include its lack of a built-in schema, limited support for data types, absence of comments within data structures, and potential security vulnerabilities when handling untrusted JSON data. While JSON excels in simplicity and interoperability, these limitations should be taken into account when using it in applications, and appropriate safeguards and validation mechanisms should be implemented to ensure data integrity and security.

3. JSON SYNTAX

The basic syntax of JSON, resembles a subset of JavaScript syntax:

1. *Data Represented in Name/Value Pairs:*

JSON represents data in the form of name/value pairs, where a name (key) is associated with a corresponding value. These name/value pairs provide a structured way to organize and store data. Names (keys) are always strings, enclosed in double quotes, and followed by a colon (`:`) to separate them from their corresponding values.

```
{
  "name": "John",
  "age": 30,
  "isStudent": false
}
```

In the example above, `"name"`, `"age"`, and `"isStudent"` are the names, while `"John"`, `30`, and `false` are their respective values.

2. *Curly Braces Hold Objects:*

In JSON, curly braces `{}` are used to enclose objects. Objects are collections of name/value pairs. The name/value pairs within an object are separated by commas (`,`). Each name is followed by a colon (`:`) to map it to its corresponding value.

```
{
  "person": {
    "name": "John",
    "age": 30
  }
}
```

In this example, the outer curly braces define an object with the name `"person"`, which contains its own name/value pairs.

3. *Square Brackets Hold Arrays:*

JSON also uses square brackets `[]` to enclose arrays. Arrays in JSON are ordered lists of values, and values within an array are separated by commas (`,`).

```
{
    "fruits": ["apple", "banana", "orange"]
}
```

Here, the `"fruits"` key maps to an array that contains a list of fruits as values.

JSON's simplicity and adherence to these syntax rules make it a versatile and widely adopted format for representing structured data. It's not only used for data interchange between systems but also for configuration files, data storage, and various other data-related tasks in modern software development. Additionally, JSON's resemblance to JavaScript syntax makes it accessible and easy for developers to work with.

Let's create a simple JavaScript program that demonstrates how to work with JSON data:

```
// Example JSON data
var jsonData = `{
   "name": "John",
   "age": 30,
   "isStudent": false,
   "fruits": ["apple", "banana", "orange"],
   "address": null
}`;
// Parsing JSON data into a JavaScript object
var data = JSON.parse(jsonData);
// Accessing JSON data
console.log("Name:", data.name);
console.log("Age:", data.age);
console.log("Is Student:", data.isStudent);
console.log("Fruits:", data.fruits);
console.log("Address:", data.address);
// Modifying JSON data
data.age = 31;
// Converting JavaScript object back to JSON
```

```
var updatedJson = JSON.stringify(data, null, 2);
// Printing the updated JSON
console.log("\nUpdated JSON:\n" + updatedJson);
```

Let's break down the code step by step to explain what each part does:

```
// Example JSON data
var jsonData = `{
   "name": "John",
   "age": 30,
   "isStudent": false,
   "fruits": ["apple", "banana", "orange"],
   "address": null
}`;
```

Here, you define a JavaScript variable `jsonData` that contains a JSON-formatted string. This JSON data represents a person's information, including their name, age, student status, a list of favorite fruits, and an address. The data is enclosed in backticks (`` ``` ``) to create a multiline string, which allows you to format the JSON data more cleanly.

```
// Parsing JSON data into a JavaScript object
var data = JSON.parse(jsonData);
```

In this part, you use `JSON.parse()` to convert the JSON-formatted string (`jsonData`) into a JavaScript object (`data`). This function parses the JSON string and creates a JavaScript object with properties that correspond to the keys in the JSON data.

```
// Accessing JSON data
console.log("Name:", data.name);
console.log("Age:", data.age);
console.log("Is Student:", data.isStudent);
console.log("Fruits:", data.fruits);
console.log("Address:", data.address);
```

Here, you access and print specific values from the JavaScript object (`data`). You use dot notation to access the object's properties, such as `data.name` to retrieve the name, `data.age`

to retrieve the age, and so on. This demonstrates how you can work with the data stored in the JavaScript object.

```
// Modifying JSON data
data.age = 31;
```

In this section, you modify one of the properties of the JavaScript object (`data`). Specifically, you update the `age` property from `30` to `31`.

```
// Printing the updated JSON
console.log("\nUpdated JSON:\n" + updatedJson);
```

This line attempts to print an updated JSON string using the `updatedJson` variable. However, there is an issue in the code: `updatedJson` is not defined in the provided code snippet. To correctly print the updated JSON string, you should use the `data` object and `JSON.stringify()` as follows:

```
// Printing the updated JSON
console.log("\nUpdated JSON:\n" + JSON.stringify(data, null, 2));
```

The `JSON.stringify()` function is used to convert the modified `data` object back into a JSON-formatted string. The `null` argument and `2` as the third argument are used for formatting the JSON string with indentation for improved readability.

4. DATA TYPES

JSON (JavaScript Object Notation) supports several data types that allow you to represent different kinds of information. Each data type in JSON has its own syntax and usage. Below, I'll explain the JSON data types in detail, along with their syntax and examples:

1. *String:*
   - Syntax: Strings in JSON are enclosed in double quotation marks (`"`).
   - Example:

```
{
  "name": "John",
  "city": "New York"
}
```

2. *Number:*
   - Syntax: Numbers in JSON can be integers or floating-point values. They are not enclosed in quotes.
   - Example:

```
{
  "age": 30,
  "price": 9.99
}
```

3. *Boolean:*
   - Syntax: JSON represents boolean values as `true` or `false` (without quotes).
   - Example:

```
{
  "isStudent": true,
  "isActive": false
}
```

4. *Object:*
   - Syntax: Objects in JSON are enclosed in curly braces `{}`. They consist of key-value pairs separated by commas. Keys must be strings enclosed in double quotes.

- Example:

```
{
  "person": {
    "name": "John",
    "age": 30
  },
  "address": {
    "city": "New York",
    "zipcode": "10001"
  }
}
```

5. *Array:*
   - Syntax: Arrays in JSON are enclosed in square brackets `[]`. They contain ordered lists of values separated by commas. Values can be of different data types, including other objects or arrays.
   - Example:

```
{
  "fruits": ["apple", "banana", "orange"],
  "numbers": [1, 2, 3, 4, 5],
  "mixed": [1, "two", false, null]
}
```

6. *Null:*
   - Syntax: JSON uses the `null` keyword to represent a null value (without quotes).
   - Example:

```
{
  "address": null
}
```

7. *Whitespace:*

- Syntax: JSON allows the use of whitespace characters (spaces, tabs, newlines) for formatting and readability but does not consider them significant for the data's meaning.
- Example:

```
{
  "name": "John",
  "age": 30
}
```

These JSON data types enable you to represent a wide range of structured information in a simple and human-readable format. JSON's versatility, combined with its compatibility across various programming languages, makes it a popular choice for data interchange and storage in modern software development.

5. PARSING IN JSON

Parsing in JSON refers to the process of converting a JSON-formatted string into a data structure that can be easily manipulated and accessed in a programming language. This process involves interpreting the text representation of JSON and creating corresponding data objects or structures within the programming environment. Parsing is a fundamental operation when working with JSON, as it allows you to work with the data in a more usable and structured form.

Tip: A data-interchange format is a text format which is used to interchange or exchange data between different platforms and operating systems. JSON is the most popular and lightweight data-interchange format for web applications.

Here's an overview of how parsing works in JSON:

1. *Input: JSON-formatted String:*

Parsing begins with a JSON-formatted string. This string can be obtained from various sources, such as an HTTP response from a web API, a file on disk, or user input. The JSON string should adhere to JSON syntax rules, including the use of double quotes for keys and string values, correct nesting of objects and arrays, and proper use of commas to separate elements.

Syntax:

JSON.parse(string, function)

Parameters: This method accepts two parameters as mentioned above and described below:

- **string:** It is a required parameter and it contains a string that is written in JSON format.
- **function:** It is an optional parameter and is used to transform results. The function called for each item.

Return Value: This method returns an object corresponding to the given JSON text.

```
let obj = JSON.parse('{"var1":"Web","var2":"Programming!"}');

console.log(obj.var1 + " " + obj.var2);
```

2. *Parsing Process:*

The JSON string is passed through a parser provided by the programming language or a dedicated JSON parsing library. The parser's job is to read the characters in the string, interpret the JSON structure, and create the corresponding data objects or structures in memory.

3. *Output: Data Structure:*

After successful parsing, the JSON data is converted into a data structure that mirrors the JSON structure. In most programming languages, this data structure is typically represented as objects, arrays, strings, numbers, booleans, and null values. The structure created depends on the structure defined in the JSON string. The output will look like this

Web Programming!

4. *Accessing and Manipulating Data:*

Once the JSON data is parsed and stored as a data structure within the programming environment, you can access and manipulate it just like any other data. You can retrieve values by accessing properties or elements, modify values, filter data, and perform various operations based on the specific language and data structures used.

Let's suppose we've received the following JSON-encoded string from a web server:

Example

```
{"name": "Peter", "age": 22, "country": "United States"}
```

Now, we can simply use the JavaScript JSON.parse() method to convert this JSON string into a JavaScript object and access individual values using the dot notation (.), like this:

Example

```
// Store JSON data in a JS variable
var json = '{"name": "Peter", "age": 22, "country": "United States"}';
// Converting JSON-encoded string to JS object
var obj = JSON.parse(json);
```

```
// Accessing individual value from JS object
alert(obj.name); // Outputs: Peter
alert(obj.age); // Outputs: 22
alert(obj.country); // Outputs: United States
```

In summary, parsing in JSON is the process of converting a JSON-formatted string into a structured data representation that can be used within a programming language. It's a crucial step when working with JSON data, as it allows developers to work with the data in a more meaningful and programmatically accessible way. Most modern programming languages provide built-in or third-party libraries for JSON parsing, simplifying the process for developers.

6. STRINGIFY

JSON.stringify() is a method in JavaScript that converts a JavaScript object or value into a JSON-formatted string. This process is known as serialization. It allows you to take structured data in JavaScript and convert it into a format that can be easily transmitted, stored, or otherwise shared with other systems or applications. Here is a detailed explanation of JSON.stringify():

Syntax:

```
JSON.stringify(value[, replacer[, space]])
```

- `value`: The JavaScript object or value to be converted into a JSON string.
- `replacer` (optional): A function or array that can be used to customize the serialization process or filter out properties to include in the resulting JSON string. It can be `null` or omitted if not needed.
- `space` (optional): A string or number used to add whitespace and indentation for formatting the resulting JSON string. It improves human readability. It can be `null`, a string, or a number (usually 0 for no extra spacing).

Example 1: Basic JSON.stringify() Usage

```
const person = {
  name: "John",
  age: 30,
```

```
  isStudent: false
};
const jsonString = JSON.stringify(person);
console.log(jsonString);
```

In this example, `person` is a JavaScript object that contains person-related information. `JSON.stringify(person)` converts this object into a JSON-formatted string, and `jsonString` contains the resulting JSON string. The output will look like:

```
{"name":"John","age":30,"isStudent":false}
```

Example 2: Using replacer Function

```
const person = {
  name: "John",
  age: 30,
  isStudent: false,
  address: {
    city: "New York",
    postalCode: "10001"
  }
};
const jsonString = JSON.stringify(person, (key, value) => {
  if (key === "postalCode") {
    return undefined; // Filter out 'postalCode' property   }
  return value;
}, 2);
console.log(jsonString);
```

In this example, a `replacer` function is used to customize the serialization process. It filters out the `"postalCode"` property from the JSON output. The `space` argument is set to `2` for better formatting:

```
{
  "name": "John",
  "age": 30,
  "isStudent": false,
  "address": {
    "city": "New York"
  }
}
```

Example 3: Using space Argument

```
const person = {
  name: "John",
  age: 30,
  isStudent: false
};
const jsonString = JSON.stringify(person, null, 4);
console.log(jsonString);
```

In this example, the `space` argument is set to `4`, which adds indentation to the resulting JSON string for improved human readability:

```
{
    "name": "John",
    "age": 30,
    "isStudent": false
}
```

In summary, `JSON.stringify()` is a crucial method for serializing JavaScript objects into JSON strings. It offers flexibility through the optional `replacer` and `space` arguments, allowing you to customize the serialization process and format the output according to your needs. This method is commonly used in scenarios involving data exchange, data storage, and configuration settings.

7. SUMMARY

The chapter provides an in-depth exploration of JSON (JavaScript Object Notation), covering several key topics like JSON, or JavaScript Object Notation, is a widely adopted data interchange format that has become a standard for representing structured data in a text-based, human-readable format. It is language-independent, making it versatile for use with various programming languages. JSON is commonly employed for data transmission in web applications, as well as for data storage and configuration files. JSON's syntax adheres to a set of rules. It uses name/value pairs, with keys (names) represented as strings enclosed in double quotes and values separated from keys by colons. Objects are enclosed in curly braces `{}`, and arrays are enclosed in square brackets `[]`. String values can contain escape characters for special characters, and numbers, booleans, and null values are represented without quotes. JSON is also flexible with whitespace, allowing for formatting and readability enhancements.

JSON supports several fundamental data types, including strings, numbers, booleans, objects, arrays, and null. These data types allow for the representation of a wide range of structured information. JSON's simplicity and flexibility make it suitable for modeling diverse data structures and scenarios. Parsing in JSON refers to the process of converting a JSON-formatted string into a structured data representation that can be easily manipulated and accessed within a programming language. JSON parsing is essential for extracting and working with data received from external sources, such as web APIs. Most programming languages provide built-in functions or libraries for parsing JSON data. JSON.stringify() is a method used to convert a JavaScript object or value into a JSON-formatted string, which is the opposite process of parsing. It allows developers to serialize structured data into a format that can be transmitted, stored, or shared with other systems or applications. JSON.stringify() provides options for customizing the serialization process, including the use of a replacer function and indentation for formatting. In summary, this chapter provides a comprehensive understanding of JSON, covering its foundational concepts, syntax rules, supported data types, parsing for data extraction, and stringification for data serialization. JSON's simplicity, human-readability, and language-independence make it a valuable tool in various domains, including web development, data interchange, and configuration management.

8. QUESTIONS

Self-Assessment Questions

SELF-ASSESSMENT QUESTIONS – 1

1. What does JSON stand for?
2. Describe the primary application of JSON in web development.
3. In JSON syntax, how are key-value pairs separated within an object?
4. Which data type in JSON is represented without quotes?
5. What is the purpose of JSON parsing?
6. How is an array represented in JSON syntax?
7. What is the primary use of JSON.stringify() in JavaScript?
8. Explain the significance of the "replacer" argument in JSON.stringify().
9. What role does the "null" keyword play in JSON data?
10. How can whitespace be used for formatting in JSON?

Terminal questions

1. Explain the significance of JSON in modern web development. How does it facilitate data exchange between a server and a client in web applications?
2. Provide a detailed breakdown of the syntax rules in JSON. Include examples for representing objects, arrays, strings, numbers, booleans, and null values.
3. Describe the primary data types supported by JSON. How does JSON's support for these data types make it suitable for representing complex structured data?
4. Walk through the process of parsing JSON data in a programming language. What are the key steps involved, and why is parsing important when working with JSON?
5. Discuss the role of the JSON.stringify() method in JavaScript. How can developers use it to serialize JavaScript objects into JSON strings, and what options are available for customizing the serialization process?
6. Explain the advantages and disadvantages of using JSON as a data interchange format. Highlight its strengths in terms of human-readability, interoperability, and efficiency, as well as its limitations related to data types and security considerations.

9. ANSWERS

Self-Assessment Questions

1. JSON stands for "JavaScript Object Notation."
2. JSON is primarily used in web development for data transmission between a server and a client, as well as for storing structured data.
3. Key-value pairs within an object in JSON are separated by colons (**:**).
4. The data type in JSON represented without quotes is a number.
5. The purpose of JSON parsing is to convert a JSON-formatted string into a structured data representation that can be easily manipulated and accessed in a programming language.
6. An array in JSON syntax is represented by enclosing values in square brackets **[]**.
7. The primary use of **JSON.stringify()** in JavaScript is to convert a JavaScript object or value into a JSON-formatted string for serialization.
8. The "replacer" argument in **JSON.stringify()** is a function or array used to customize the serialization process or filter out properties to include in the resulting JSON string.
9. In JSON, the "null" keyword represents a null value, indicating the absence of a value.
10. Whitespace can be used for formatting in JSON to improve human readability, although it is not significant for the data's meaning.

Terminal questions

1. Refer Section 2
2. Refer Section 3
3. Refer Section 4
4. Refer Section 5
5. Refer Section 6
6. Refer Section 2